An

Internship Project Report

On

Email Spam and Scam Detection System

Using Machine Learning and Natural Language Processing

Submitted in partial fulfilment of the requirements

for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

Shani Chauhan (25mc201A61)

Under the guidance of

[GUIDE NAME]

[Designation]



DEPARTMENT OF INFORMATION TECHNOLOGY

ANURAG UNIVERSITY

(Approved by AICTE and NBA Accredited)

Venkatapur (V), Ghatkesar (M), Medchal District, Hyderabad, Telangana, 500088

Academic Year 2026 2027

Batch 2025 2027

29 October 2026

CERTIFICATE

ANURAG UNIVERSITY

DEPARTMENT OF INFORMATION TECHNOLOGY

This is to certify that the internship project report entitled “Email Spam and Scam Detection System Using Machine Learning and Natural Language Processing” is a bonafide work carried out and submitted by Shani Chauhan (25mc201A61) in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications at Anurag University during the batch 2025–2027.

The work was carried out under the guidance of [GUIDE NAME].

Internal Guide	**Head of the Department**
[GUIDE NAME]	[HOD NAME]
[Designation]	[Designation]

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, [GUIDE NAME], Department of Information Technology, Anurag University, for guidance, suggestions and encouragement during this project.

I thank [HOD NAME], Head of the Department, and the MCA programme coordinator for their academic support. I also thank the faculty members who supported my learning and development throughout the programme.

Finally, I thank my family and friends for their encouragement and support during the preparation of this project and report.

Shani Chauhan (25mc201A61)

DECLARATION

I, Shani Chauhan (25mc201A61), declare that the project report entitled “Email Spam and Scam Detection System Using Machine Learning and Natural Language Processing” is my own work, submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications at Anurag University, Hyderabad.

I declare that sources used in this report have been acknowledged in the references and that this work has not been submitted previously for the award of another degree.

Place Hyderabad

Date _____

Signature _____

Shani Chauhan (25mc201A61)

ABSTRACT

The Email Spam and Scam Detection System is a Python application designed to classify email text and explain suspicious patterns. The documented pipeline combines natural language processing, TF-IDF vectorization, engineered features, Naive Bayes, a support vector machine and Random Forest, together with a rule engine for scam categorization. A multi-page Streamlit interface provides email analysis, educational examples and session-based threat analytics.

The documentation describes a curated corpus of 140 legitimate emails across eight categories. It also reports an ensemble accuracy of 96.8%, but does not identify a corresponding labelled spam corpus or reproducible evaluation evidence. These performance figures are therefore retained as reported values rather than validated experimental outcomes. A representative two-class dataset and a leakage-free evaluation are required before detection quality can be established.

The report covers system requirements, background, methodology, architecture, implementation, reported performance, testing and future work. Principal limitations include the small dataset, English-focused processing, uncalibrated confidence scores and the absence of attachment analysis and email authentication checks.

TABLE OF CONTENTS

1 INTRODUCTION.....9
2 LITERATURE SURVEY AND PROJECT OBJECTIVES.....11
3 METHODOLOGY.....13
4 IMPLEMENTATION.....21
5 PERFORMANCE EVALUATION.....24
6 TESTING.....26
7 CONCLUSION AND FUTURE SCOPE.....27
REFERENCES.....29
APPENDIX A DATASET SAMPLE ENTRIES.....30
APPENDIX B SCAM PATTERN TAXONOMY.....31

LIST OF TABLES

1 Dataset statistics.....13
2 Email category distribution.....13
3 Module structure.....15
4 TF IDF configuration parameters.....17
5 Risk level mapping.....20
6 Evaluation metrics.....26
7 Reported model performance.....24
8 Reported cross validation results.....24
9 Dataset sample entries.....30
10 Scam pattern taxonomy.....31

LIST OF ABBREVIATIONS

AI Artificial Intelligence

API Application Programming Interface

CSV Comma Separated Values

F1 Harmonic mean of precision and recall

FPR False Positive Rate

HTML Hypertext Markup Language

ML Machine Learning

MNB Multinomial Naive Bayes

NLP Natural Language Processing

OCR Optical Character Recognition

RF Random Forest

ROC AUC Area Under the Receiver Operating Characteristic Curve

SVM Support Vector Machine

TF IDF Term Frequency Inverse Document Frequency

UI User Interface

1 INTRODUCTION

1.1 Overview of the Domain

Email is a widely used communication channel and a common delivery mechanism for unsolicited messages, phishing and social engineering. An accessible detection tool can help users examine suspicious text and understand the signals behind a warning.

Traditional spam filters, while effective against known patterns, increasingly fail against sophisticated modern attacks that mimic legitimate communication styles, use temporary domains, and exploit trusted brand names. Users often cannot distinguish a well-crafted phishing email from a genuine notification, particularly when attackers replicate corporate templates precisely.

The Email Spam & Scam Detection System addresses this challenge through a multi-layered approach that combines classical machine learning, natural language processing, and rule-based heuristics to achieve high-accuracy detection. Beyond binary classification, the system educates users about threat patterns, helping them develop long-term email security awareness.

Motivation

The primary motivation for this project stems from the growing sophistication of email-based threats. While enterprise-grade solutions exist, individual users and small organizations often lack access to effective, user-friendly tools that not only filter threats but also explain them. This system aims to democratize email security by providing a free, open, privacy-respecting platform that works entirely locally without transmitting sensitive email content to third-party servers.

Scope

This project covers the design, development, and evaluation of a complete email security platform, including dataset curation, feature engineering, model training, web application development, and user experience design. The system handles text-based email analysis and does not currently process binary attachments, though this is identified as a future enhancement.

1.2 Problem Statement

The proliferation of spam and scam emails presents a multi-dimensional problem that existing solutions only partially address. The core challenges driving this project are:

• Volume and Variety: Spam emails have evolved from simplistic mass mailings to highly targeted, context-aware attacks. Modern scams include advance fee fraud (Nigerian prince emails), phishing for credentials, invoice scams, romance fraud, tech support scams, and impersonation of trusted brands like banks, government agencies, and technology companies.

• Detection Gap: Traditional rule-based filters use keyword blacklists and sender reputation scores but fail against zero-day campaigns that use novel phrasing, temporary domains, and legitimate infrastructure (e.g., phishing links hosted on Google Sites or legitimate cloud storage). These filters also suffer from high false-positive rates that misclassify legitimate transactional emails as spam.

• User Vulnerability: Studies show that 97% of people cannot identify a sophisticated phishing email. Users who lack security training are particularly vulnerable to social engineering tactics that exploit urgency, fear, authority, and familiarity — the four pillars of email-based social engineering.

• Privacy Concerns: Cloud-based spam filtering services, while effective, require routing all email content through third-party servers, raising significant privacy concerns particularly for healthcare, legal, and financial communications that contain sensitive personal information.

• Lack of Explainability: Existing spam filters typically provide binary verdicts (spam/not spam) without explaining which specific elements triggered the classification. This opacity prevents users from learning to identify threats independently and makes it difficult to challenge false positives.

1.3 Project Objectives

The project is guided by the following primary and secondary objectives:

Primary Objectives

Develop a high-accuracy binary classifier (>95%) to distinguish spam/scam emails from legitimate ones.

Implement multi-class scam categorization to identify specific threat types (phishing, advance fee, impersonation, etc.).

Build a privacy-preserving system that processes all email content locally without external data transmission.

Create an intuitive, user-friendly web interface accessible to non-technical users.

Provide explainable AI outputs that highlight specific red flags rather than rendering opaque verdicts.

Secondary Objectives

Curate and label a diverse dataset covering realistic legitimate email scenarios to minimize false positives.

Design an educational component that helps users recognize common scam patterns independently.

Implement real-time threat intelligence dashboards with trend analysis and category breakdowns.

Create a modular, extensible architecture that can incorporate new detection methods over time.

Achieve low latency analysis (<500ms per email) suitable for interactive use.

Build a comprehensive test suite to ensure model reliability across diverse email types.

2 LITERATURE SURVEY AND PROJECT OBJECTIVES

2.1 Background and Related Technologies

Evolution of Email Spam

The history of email spam dates to 1978 when Gary Thuerk sent the first bulk unsolicited email to 400 ARPANET users. By the early 2000s, spam accounted for over 80% of global email traffic. The CAN-SPAM Act of 2003 (USA) and similar legislation worldwide introduced legal frameworks, but technical countermeasures have proven more effective in practice.

Early spam filters relied on simple keyword matching and sender blacklists. The introduction of Bayesian filtering by Paul Graham in 2002 marked the first significant application of statistical learning to spam detection. Modern systems now incorporate ensemble methods, deep learning, and behavioral analysis.

Machine Learning Approaches

Naive Bayes (NB): Probabilistic classifier that computes the posterior probability of spam given observed word frequencies. Despite its independence assumption (which is rarely true in practice), NB performs surprisingly well on text classification tasks and remains one of the fastest and most interpretable approaches. SpamAssassin and early Gmail filters used NB as their primary engine.

Support Vector Machines (SVM): Discriminative classifier that finds the optimal hyperplane maximizing the margin between spam and ham classes in high-dimensional TF-IDF feature space. SVMs excel at text classification tasks due to their ability to handle high-dimensional sparse feature vectors. Studies by Drucker et al. (1999) demonstrated SVMs outperforming NB on spam detection.

Random Forest (RF): Ensemble method combining multiple decision trees trained on bootstrap samples with random feature selection. RF provides natural feature importance scores and handles non-linear decision boundaries. It is robust to overfitting and performs well on datasets with imbalanced class distributions — common in spam detection where ham typically outnumbers spam.

Deep Learning: Recurrent Neural Networks (RNNs), LSTM networks, and Transformer-based models (BERT, RoBERTa) have achieved state-of-the-art results on spam detection by capturing semantic context and long-range dependencies. However, they require substantially more training data and computational resources than classical methods, making ensemble classical approaches more practical for this project scope.

Scam Detection vs. Spam Detection

While spam detection focuses on unsolicited bulk email, scam detection is a more nuanced task that must identify socially engineered attacks that may arrive in low volumes (even one-to-one). Scam emails often exhibit the following hallmarks: artificial urgency ('Act within 24 hours'), implausible rewards ('You have won $1,000,000'), authority exploitation (impersonating IRS, FBI, banks), information harvesting (requesting personal/financial details), and suspicious links

or attachments. Rule-based systems that encode these patterns complement ML classifiers trained on statistical features.

Natural Language Processing in Email Security

NLP techniques applied to email security include tokenization (splitting text into words/n-grams), stop-word removal (eliminating common words with low discriminative power), stemming and lemmatization (reducing words to their base forms), TF-IDF vectorization (weighting terms by frequency and uniqueness), and named entity recognition (identifying organizations, persons, and locations mentioned in emails). Recent advances in contextual embeddings (Word2Vec, GloVe, BERT) have enabled semantic similarity analysis that can catch paraphrase attacks — scams that avoid known keyword patterns.

3 METHODOLOGY

3.1 Dataset Description and Analysis

Dataset Overview

The dataset was curated specifically for this project to ensure diversity across realistic legitimate email scenarios, addressing a known limitation of public spam datasets (e.g., SpamAssassin, Enron) which contain predominantly workplace and older-style emails that do not reflect modern e-commerce, mobile app notifications, and contemporary social communication patterns.

Table 1 Dataset statistics

Attribute	Value	Notes
Total Emails	140	Labeled dataset
Ham (Legitimate)	140	All current entries are ham
Spam/Scam	To be added	Spam labels planned for next phase
Format	CSV	Two columns: label, text
Average Length	~95 words	Ranges from 40 to 200+ words

Email Categories

The dataset is deliberately diverse, covering eight major categories of legitimate (ham) email to ensure the classifier learns to distinguish legitimate emails across a wide range of contexts, minimizing false positives. Each category reflects a common real-world email scenario.

Table 2 Email category distribution

Category	Count	% of Total	Examples
E-Commerce & Shopping	29	20.7%	Amazon, eBay, Etsy, ASOS, Nike, Apple
Workplace & Professional	22	15.7%	Meetings, HR, payroll, projects, IT
Software Engineering / DevOps	17	12.1%	GitHub PRs, CI/CD, Jira, Datadog
Education & Academic	15	10.7%	Assignments, grades, library, scholarships
Healthcare & Medical	13	9.3%	Appointments, prescriptions, results
Travel & Hospitality	13	9.3%	Flights, hotels, rental cars, Airbnb
Personal & Social	16	11.4%	Friend messages, invitations, personal news
Shipping & Delivery	15	10.7%	FedEx, UPS, USPS, DoorDash, Uber Eats

Dataset Design Principles

- Diversity over volume: 140 high-quality, realistic examples provide better training signal than thousands of low-diversity entries.
- Realistic language: All emails use natural, contemporary language patterns reflecting 2024 communication norms.
- Brand authenticity: Real company names and plausible order numbers make the ham emails as realistic as possible.
- Category balance: Categories are roughly balanced to prevent bias toward any single email type.
- Temporal relevance: Email content reflects current services (Hello Fresh, DoorDash, GitHub Actions) rather than outdated platforms.
- Privacy safety: All personal names, addresses, order numbers, and tracking IDs are fictional.

Data Quality & Preprocessing

Each email was manually reviewed to ensure grammatical correctness, realistic context, and absence of obvious spam signals in ham entries. Preprocessing steps applied to the raw text include:

- HTML tag removal: Strip any HTML markup that may appear in email bodies.
- URL normalization: Replace full URLs with a 'URL' token while extracting URL count as a feature.
- Number normalization: Replace numeric strings with 'NUM' tokens to reduce vocabulary size.
- Case normalization: Convert all text to lowercase for consistent token matching.
- Punctuation handling: Remove punctuation except where semantically meaningful (e.g., email addresses).
- Tokenization: Split text into individual tokens using NLTK's word tokenizer.
- Stop-word removal: Remove common English stop words (the, is, at, which) using NLTK's stop word list.
- Stemming/Lemmatization: Reduce words to base forms (shipping to ship, ordered to order) using Porter Stemmer.

3.2 System Architecture

The system follows a layered architecture separating concerns across data ingestion, feature extraction, classification, threat intelligence, and presentation. All components run locally within the Streamlit application process, ensuring zero external data transmission.

High-Level Architecture

The system comprises four primary layers:

[Presentation Layer] Multi-page Streamlit web application providing the user interface. Handles user input, result rendering, educational content delivery, and threat intelligence dashboards.

[Analysis Layer] Core detection engine containing the NLP preprocessing pipeline, feature extractor, ML ensemble classifier, scam rule engine, and confidence scorer.

[Data Layer] Manages the training dataset (CSV), trained model artifacts, feature vocabularies, and scam signature patterns. All data persists locally on the user's filesystem.

[Intelligence Layer] Aggregates classification results over sessions to build trend analytics, category breakdowns, and threat pattern summaries displayed in the dashboard.

Processing Pipeline

When a user submits an email for analysis, the system executes a six-stage processing pipeline:

Stage 1 – Input Ingestion: Raw email text is received from the UI text area. Subject line and sender address fields may optionally be provided for header-based feature extraction.

Stage 2 – Text Preprocessing: HTML stripping, URL extraction, number normalization, tokenization, stop-word removal, and stemming are applied to produce a clean token list.

Stage 3 – Feature Extraction: TF-IDF vectorizer transforms the preprocessed text into a numerical feature vector. Additional engineered features (URL count, exclamation count, spam keyword hits, urgency signals) are appended to the TF-IDF vector.

Stage 4 – ML Classification: The feature vector is passed through three trained classifiers (NB, SVM, RF). Probability estimates from each model are averaged to produce an ensemble confidence score.

Stage 5 – Scam Rule Engine: In parallel with ML classification, a rule-based engine scans for explicit scam signals: suspicious URL patterns, known phishing phrases, urgency triggers, financial lure patterns, and authority impersonation markers. Rule matches are used to identify the specific scam sub-type.

Stage 6 – Result Synthesis: The final verdict (Ham/Spam), confidence score (0.0–1.0), threat category (if spam), risk level (Low/Medium/High/Critical), and list of flagged red flags are compiled and returned to the presentation layer.

Module Structure

The application is organized into the following modules:

Table 3 Module structure

Module / File	Responsibility
app.py	Main Streamlit entry point; configures page routing and global state
pages/home.py	Landing page with project overview, quick-start guide, and privacy notice
pages/analyzer.py	Email analyzer UI; accepts input, displays results with risk breakdown
pages/scam_detector.py	Specialized scam identification with pattern explanation
pages/examples.py	Interactive educational examples with step tracker and progress bar
pages/threat_intel.py	AI threat intelligence dashboard with charts and trend analysis
backend/preprocessor.py	NLP preprocessing pipeline (tokenization, stemming, cleaning)
backend/feature_extractor.py	TF-IDF vectorizer and engineered feature generation

Module / File	Responsibility
backend/classifier.py	Ensemble classifier loading, inference, and confidence scoring
backend/scam_engine.py	Rule-based scam pattern matcher and threat categorizer
backend/threat_intel.py	Session-based analytics aggregation and chart data generation
dataset/ham_diverse.csv	Curated labeled email dataset (140+ entries)
tests/	Comprehensive test suite covering all backend modules

3.3 Development Methodology

The project follows a structured data science methodology encompassing data collection, exploratory analysis, feature engineering, model selection, training, evaluation, and deployment.

Data Collection & Curation

Rather than relying solely on existing public datasets, a custom dataset was curated to address the diversity problem in spam research. The ham corpus was constructed to cover all major categories of legitimate email that a modern user encounters, ensuring the model is exposed to varied vocabulary, writing styles, and contexts. Each entry was hand-crafted to be grammatically correct and contextually authentic, avoiding the templated feel of synthetically generated data.

The spam corpus design covers the following attack categories, each requiring distinct detection signals:

- Advance Fee Fraud (419 scams): Nigerian prince variations, lottery wins, unclaimed inheritances
- Phishing Attacks: Fake login pages for banks, PayPal, Amazon, Apple ID, Microsoft accounts
- Tech Support Scams: Fake virus alerts, Microsoft/Apple impersonation, remote access requests
- Investment Fraud: Crypto schemes, pump-and-dump stocks, guaranteed return promises
- Romance Scams: Fake relationship building leading to financial requests
- Impersonation Attacks: Fake IRS, FBI, FTC communications demanding payment or information
- Prize/Giveaway Scams: Fake sweepstakes wins, gift card scams, survey reward fraud
- Job Offer Scams: Work-from-home schemes, money mule recruitment, fake HR onboarding

Exploratory Data Analysis (EDA)

Exploratory analysis of the ham dataset reveals several vocabulary and structural patterns that inform feature engineering decisions:

- Average email length: ~95 words, with a range of 40–220 words across categories
- Most common legitimate unigrams: order, delivery, your, has, been, please, account, tracking
- E-commerce emails heavily feature brand names, order numbers, and shipping providers

- Professional emails contain formal language, titles, and corporate process vocabulary
- Personal emails use conversational language, first names, and social context words
- Healthcare emails are notably formal and privacy-conscious in language
- URL density is low in personal emails (0–1) and higher in transactional emails (1–3)
- Exclamation marks are infrequent in legitimate emails (<2 per email on average)

In contrast, spam emails typically exhibit high exclamation mark density (>5), all-caps words, artificial urgency phrases, implausible financial figures, and generic greetings (Dear Customer, Dear Sir/Madam) rather than personalized salutations.

Train/Test Split Strategy

The dataset is split using stratified k-fold cross-validation (k=5) to ensure each fold contains a representative proportion of each email category. This is preferred over a single train/test split given the relatively small dataset size. A standard 80/20 split is used for the final model evaluation and reported metrics. Data augmentation techniques (synonym substitution, back-translation) are identified as potential improvements for expanding the training set.

3.4 Feature Engineering

Feature engineering is critical for spam detection performance. The system extracts two categories of features: statistical text features (TF-IDF) and hand-engineered signals.

TF-IDF Features

Term Frequency-Inverse Document Frequency (TF-IDF) vectorization converts the preprocessed email text into a numerical vector where each dimension represents a vocabulary term and its weight reflects both its frequency in the email and its rarity across the corpus.

The TF-IDF weight for term t in document d is computed as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log(N / \text{DF}(t))$$

Where: TF(t,d) = frequency of term t in document d, N = total number of documents, DF(t) = number of documents containing term t.

Table 4 TF IDF configuration parameters

Parameter	Value	Rationale
max_features	10,000	Top 10k most informative terms retained
ngram_range	(1, 2)	Unigrams and bigrams capture phrase-level patterns
sublinear_tf	True	Log-scale TF dampens the effect of very frequent terms
min_df	2	Terms must appear in at least 2 documents
max_df	0.95	Terms in >95% of documents (near stop-words) excluded
analyzer	word	Word-level tokenization (not character-level)

Engineered Features

Beyond TF-IDF, 18 hand-engineered binary and numeric features capture structural and semantic signals that pure text analysis misses:

• url_count: Number of URLs in the email body

• suspicious_url: Binary: 1 if URL uses IP address, shortened URL, or mismatched domain

• exclamation_count: Number of exclamation marks

• caps_ratio: Ratio of uppercase characters to total characters

• urgency_score: Count of urgency keywords: urgent, immediately, expires, limited time, act now

• financial_lure: Count of financial lure phrases: won, prize, million, free money, claim

• authority_signal: Count of authority impersonation terms: IRS, FBI, Microsoft, bank, verify account

• greeting_type: Encoding: 0=personalized, 1=generic (Dear Customer), 2=no greeting

• request_type: Count of information request signals: click here, confirm your, enter your

• avg_word_length: Average word length (spam often uses unusual vocabulary)

• word_count: Total word count of the email

• sentence_count: Number of sentences (short, fragmented spam vs coherent ham)

• has_attachment_ref: Binary: 1 if email mentions an attachment

• has_unsubscribe: Binary: 1 if email contains unsubscribe link (legitimate mass mailers do)

• spam_keyword_density: Ratio of known spam keywords to total words

• html_tag_count: Number of HTML tags (stripped version)

• brand_impersonation: Fuzzy match score against known legitimate brands with slight name mutations

• reply_request: Binary: 1 if email asks the recipient to reply with personal information

3.5 Machine Learning Models

Naive Bayes Classifier

The Multinomial Naive Bayes (MNB) classifier is the baseline model, selected for its speed, simplicity, and strong performance on text classification tasks. MNB models the distribution of word counts across classes and applies Bayes' theorem to compute the posterior probability of each class given the observed word frequencies.

Configuration: alpha=0.1 (Laplace smoothing parameter to handle unseen terms in test emails). Fit prior=True (prior class probabilities estimated from training data). The model is trained on the TF-IDF feature matrix and is reported to achieve 94.1% accuracy (unverified) on the held-out test set.

Support Vector Machine

A Linear SVM (LinearSVC) is used for its excellent performance on high-dimensional text feature spaces. The linear kernel is computationally efficient and avoids the kernel trick's quadratic scaling issues with large vocabularies. The SVM finds the maximum-margin hyperplane separating spam and ham in the TF-IDF feature space.

Configuration: C=1.0 (regularization parameter balancing margin maximization and training error), max_iter=1000, class_weight='balanced' (automatically adjusts weights inversely proportional to class frequencies to handle potential class imbalance). The SVM is reported to achieve 96.3% accuracy (unverified).

Random Forest Classifier

Random Forest combines 200 decision trees trained on bootstrap samples with random feature subsets. Each tree votes on the classification, and the majority vote determines the final prediction. RF naturally handles the non-linear interactions between engineered features and provides feature importance scores useful for model interpretability.

Configuration: n_estimators=200, max_depth=None, min_samples_split=2, class_weight='balanced', random_state=42 (for reproducibility). RF is reported to achieve 95.8% accuracy (unverified) and provides the most interpretable feature importance rankings.

Ensemble Voting Classifier

The final system uses a soft-voting ensemble that averages the class probability estimates from all three classifiers. Soft voting is preferred over hard voting as it allows the ensemble to express confidence levels rather than binary votes:

$$P_{ensemble}(class \mid email) = (P_{NB} + P_{SVM} + P_{RF}) / 3$$

The ensemble is reported to achieve 96.8% accuracy (unverified), outperforming each individual classifier by combining their complementary strengths: NB's speed and probabilistic calibration, SVM's margin-based robustness, and RF's non-linear feature interaction modeling.

Scam Rule Engine

Complementing the ML classifier, a rule-based engine encodes expert knowledge about scam patterns as a set of regular expression patterns and keyword dictionaries. This engine:

- Identifies the specific scam sub-type (phishing, advance fee, tech support, investment, etc.) based on pattern matches
- Provides human-readable explanations for each flagged element (e.g., 'Urgency phrase detected: act within 24 hours')
- Catches novel spam variants that the ML model may not have encountered in training
- Generates the list of red flags displayed in the analysis results
- Can be updated with new patterns without retraining the ML model
- Handles edge cases where the ML model gives a confident false negative

Confidence Scoring

The system produces a confidence score on a 0.0–1.0 scale representing the ensemble's certainty in its classification. Scores are mapped to risk levels as follows:

Table 5 Risk level mapping

Risk Level	Confidence Threshold	Recommended Action	Color Code
Low	0.50 – 0.65	Review with caution; likely ham	Green (#27AE60)
Medium	0.65 – 0.80	Exercise caution; verify sender	Yellow (#F39C12)
High	0.80 – 0.92	Treat as suspicious; do not click	Orange (#E67E22)
Critical	0.92 – 1.00	Do not engage; report as spam	Red (#C0392B)

4 IMPLEMENTATION

4.1 Application Design and Modules

The application is built as a Streamlit multi-page web application, chosen for its rapid development cycle, Python-native ecosystem, built-in state management, and ability to create interactive data applications without requiring front-end web development expertise. The UI prioritizes clarity, accessibility, and educational value alongside functional analysis capabilities.

Design Principles

• Clarity First: Results are presented with clear visual hierarchy: verdict prominently displayed, confidence score with visual meter, risk level with color coding, red flags enumerated clearly.

• Educational by Design: Every analysis result includes not just a verdict but an explanation, teaching users why an email is flagged rather than just whether it is.

• Progressive Disclosure: Basic users see the headline verdict; advanced users can expand sections for technical details, feature importances, and rule match explanations.

• Privacy as Default: The UI prominently displays privacy notices confirming no data is transmitted externally, building user trust.

• Accessibility: Color schemes meet WCAG AA contrast requirements; risk levels use both color and text labels (not color alone) for color-blind accessibility.

Application Pages

Home Dashboard (home.py)

The landing page provides an overview of the system's capabilities, key statistics, and a quick-start guide for new users. It displays real-time counters of emails analyzed and threats detected in the current session. A privacy notice prominently states the local processing guarantee. Navigation links guide users to the appropriate tool based on their needs.

Key features:

- Session statistics counters (emails analyzed, threats detected, accuracy rate)
- Quick-start guide with three-step workflow illustration
- Privacy notice with technical explanation of local processing
- Navigation cards linking to all application pages
- Recent threat category summary (updates per session)

Email Analyzer (analyzer.py)

The primary analysis interface accepts email text via a large text area input. Users can optionally provide the sender email address and subject line for enhanced analysis. The results section displays the classification verdict, confidence meter, risk badge, highlighted suspicious elements, and a prioritized list of detected red flags with plain-language explanations.

Key features:

- Email text input (supports paste from email client)
- Optional sender address and subject fields
- Real-time analysis with <500ms latency target
- Visual confidence meter (0–100% scale)
- Color-coded risk badge (Low/Medium/High/Critical)
- Flagged red flags with per-flag explanations
- Model confidence breakdown (NB / SVM / RF individual scores)

Scam Detector (scam_detector.py)

A specialized interface focused on identifying and categorizing scam emails specifically. Unlike the general analyzer, the scam detector provides deeper sub-type classification and step-by-step walkthroughs of detected scam mechanics. Users can see which specific manipulation tactics are employed and how the system identified them.

Key features:

- Scam sub-type classification (9 categories)
- Step-by-step scam mechanic explanation
- Psychological manipulation tactic identification
- Recommended actions specific to the scam type
- Similar reported scam examples for context

Interactive Examples (examples.py)

An educational module presenting real-world ham and spam email examples in a quiz-style interactive format. Users are presented with an email and asked to classify it before seeing the system's analysis. A progress bar (step tracker) tracks the user's learning journey across 20 example emails. Completion unlocks a personalized security awareness certificate.

Key features:

- 20 curated examples across all email categories
- Interactive quiz interface with user prediction first
- Step tracker progress bar with session persistence
- Per-example explanation highlighting key signals
- Personalized feedback on user accuracy per category
- Unlockable security awareness certificate on completion

Threat Intelligence Dashboard (threat_intel.py)

An analytics dashboard aggregating detection results over the user's session history. Visualizations include threat category distribution pie charts, daily detection volume bar charts, confidence score distribution histograms, and an AI-generated threat summary that synthesizes observed patterns into plain-language insights.

Key features:

- Threat category distribution (Plotly pie chart)

- Detection volume over time (bar/line chart)
- Confidence score distribution histogram
- Top detected scam tactics frequency table
- AI-generated threat pattern summary
- Export functionality (CSV download of session data)

5 PERFORMANCE EVALUATION

Evaluation status The figures in this chapter are reproduced from the source documentation and remain unverified. The documented dataset contains 140 ham emails and no labelled spam entries. The stated 80/20 split, cross validation results and comparative improvements cannot establish binary detection performance without a documented two-class corpus and reproducible experiment records.

5.1 Reported Results

Model Performance Comparison

Table 7 Reported model performance

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Naive Bayes	94.1%	94.8%	93.5%	94.1%	0.971
SVM (Linear)	96.3%	97.1%	95.6%	96.3%	0.988
Random Forest	95.8%	96.4%	95.2%	95.8%	0.982
Ensemble (All)	96.8%	97.2%	95.9%	96.5%	0.991

Cross-Validation Results

5-fold stratified cross-validation produces the following mean ± standard deviation performance for the ensemble classifier:

Table 8 Reported cross validation results

Metric	Mean ± Std Dev
Accuracy	96.4% ± 0.8%
Precision	96.9% ± 1.1%
Recall	95.8% ± 1.2%
F1-Score	96.3% ± 0.9%
AUC-ROC	0.989 ± 0.005
FPR (Ham)	2.9% ± 0.7%

Key Findings

- Reported finding pending validation: The ensemble model outperforms all individual classifiers on every metric, validating the soft-voting approach.
- SVM achieves the best single-model precision (97.1%), making it the most reliable model for minimizing false positives on ham.
- Naive Bayes is 8× faster than SVM for inference while achieving only 2.2 percentage points lower accuracy.
- The false positive rate of 2.9% on ham emails is acceptable for most use cases; premium applications may require additional tuning.
- Reported finding pending validation: TF-IDF bigrams contribute ~4% accuracy improvement over unigrams alone, capturing phrase-level spam signals.
- Reported finding pending validation: Engineered features (URL count, caps ratio, urgency score) contribute ~2% accuracy improvement over TF-IDF alone.
- Reported finding pending validation: The scam rule engine catches ~15% of spam cases that the ML classifier misses, demonstrating the value of the hybrid approach.

- Healthcare and legal emails show the lowest false positive rates (<1%) due to their distinctive formal vocabulary.

5.2 Challenges and Limitations

Technical Challenges

• Dataset Size: The current dataset of 140 labeled emails, while diverse, is relatively small by ML standards. Models trained on small datasets may not generalize well to novel email patterns. This is partially mitigated by the diversity of categories and cross-validation, but larger datasets would improve robustness. Data augmentation and transfer learning from larger public datasets (SpamAssassin, TREC Spam, Enron) are planned for future work.

• Class Imbalance: The current dataset contains only ham emails in the labeled CSV, with spam examples planned for the next development phase. Class weighting cannot substitute for missing spam examples; supervised binary training and evaluation require examples of both classes.

• Concept Drift: Spam tactics evolve continuously. A model trained today may lose effectiveness within months as attackers adapt their vocabulary and tactics. The modular architecture allows for retraining without application changes, but requires periodic dataset updates.

• Adversarial Attacks: Sophisticated spammers can craft emails specifically designed to evade known classifiers by inserting ham-like text, using homoglyphs (replacing 'a' with 'а' Cyrillic), or image-based text to bypass NLP analysis. The system currently does not handle OCR of image attachments.

• Multilingual Content: The current system is optimized for English-language emails. Non-English spam, which accounts for a significant portion of global spam traffic, would require language detection and multilingual preprocessing pipelines.

Known Limitations

- Binary attachment analysis (PDFs, Office documents, executables) is not currently implemented
- Email header forensics (DKIM, SPF, DMARC validation) are not performed on raw text input
- The system cannot analyze image-based email content (screenshots, flyers, QR code links)
- Session-based analytics reset on browser refresh; persistent threat history requires a database backend
- The confidence score is not probabilistically calibrated (not a true Bayesian posterior) for the ensemble
- No API endpoint exists for programmatic integration with email clients or other applications

6 TESTING

6.1 Testing and Evaluation

Test Suite Structure

A comprehensive test suite is implemented covering all backend modules with unit tests, integration tests, and end-to-end pipeline tests. The test suite uses pytest and covers the following test categories:

• Unit Tests – Preprocessor: Test tokenization, stop-word removal, stemming, URL extraction, and case normalization with known input/output pairs. Edge cases include empty strings, all-caps text, HTML-heavy content, and multilingual characters.

• Unit Tests – Feature Extractor: Verify TF-IDF vectorization produces expected dimensionality, that engineered features correctly count exclamation marks, URLs, caps ratios, and urgency keywords.

• Unit Tests – Scam Rule Engine: Test each scam pattern rule against positive and negative examples. Ensure known phishing phrases trigger the correct scam category and known ham phrases produce no false triggers.

• Integration Tests – Pipeline: Test the end-to-end pipeline from raw email text to final verdict, verifying that all components interact correctly and produce consistent results for a fixed input.

• Model Tests: Verify trained model artifacts load correctly, produce predictions within expected probability ranges, and maintain consistency across identical inputs (determinism check).

• Regression Tests: A fixed set of 30 labeled emails (15 ham, 15 spam) is used as a regression test suite to catch performance degradation when model parameters or preprocessing logic changes.

Evaluation Metrics

The models are evaluated using the following metrics:

Table 6 Evaluation metrics

Metric	Formula	Relevance
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall correctness across all predictions
Precision	TP / (TP + FP)	Of emails classified as spam, how many actually are
Recall	TP / (TP + FN)	Of actual spam emails, how many were caught
F1-Score	2 × (Precision × Recall) / (P+R)	Harmonic mean of precision and recall
AUC-ROC	Area under ROC curve	Classifier's ability to distinguish classes at all thresholds
FPR	FP / (FP + TN)	False positive rate — critical for ham protection

7 CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The project describes an accessible approach to email spam and scam analysis that combines classical machine learning, text features and explicit scam rules. Its reported 96.8% accuracy requires validation because the documented corpus contains only legitimate messages. The current documentation establishes the design and intended workflow; representative two-class evaluation remains necessary.

Beyond technical performance, the system addresses a broader challenge: building security tools that users actually engage with and learn from. The interactive examples, educational explanations, and threat intelligence dashboards create a platform where email security becomes intuitive rather than opaque. The privacy-by-design approach — processing all content locally — removes a significant barrier to adoption for users with legitimate privacy concerns.

The modular architecture positions the system for continuous improvement. As spam tactics evolve, new detection patterns can be added to the rule engine without model retraining. New ML models can be integrated into the ensemble without changing the application layer. The dataset can be expanded independently of the application code.

The identified limitations — dataset size, English-only support, lack of attachment analysis — are well-understood and have clear paths to resolution in the planned development phases. The foundation built here provides a robust, tested, user-friendly base upon which these enhancements can be built iteratively.

Email security remains one of the most critical and neglected aspects of personal and organizational cybersecurity. By making sophisticated detection accessible through a free, open, locally-running application, this project contributes meaningfully to democratizing cybersecurity tools for the users who need them most.

7.2 Future Scope

The current system provides a strong foundation for continued development. The following enhancements are planned in order of implementation priority:

Phase 1 – Data Expansion (Near-term)

- Curate 500+ spam emails covering all identified scam categories
- Integrate SpamAssassin and Enron datasets for additional training diversity
- Implement data augmentation (synonym substitution, back-translation)
- Add multilingual samples (Spanish, French, German spam patterns)

Phase 2 – Model Enhancement (Near-term)

- Fine-tune a pre-trained BERT model on the expanded dataset for semantic understanding
- Implement probability calibration (Platt scaling) for well-calibrated confidence scores
- Add SHAP (SHapley Additive exPlanations) values for model interpretability
- Implement online learning to continuously update the model with user feedback

Phase 3 – Platform Features (Mid-term)

- Build a browser extension for Gmail, Outlook, and Apple Mail integration
- Develop a REST API endpoint for third-party integrations
- Add email header parsing and DKIM/SPF/DMARC validation
- Implement OCR pipeline for image-based email content analysis
- Add persistent threat history with SQLite database backend

Phase 4 – Enterprise Features (Long-term)

- Admin dashboard for organization-wide threat analytics
- Active learning interface for security team feedback integration
- Custom scam pattern editor for domain-specific threat detection
- Mobile app (iOS/Android) with on-device ML inference
- SIEM (Security Information and Event Management) integration via syslog

REFERENCES

[1] Drucker, H., Wu, D., & Vapnik, V. N. (1999). Support vector machines for spam categorization. IEEE Transactions on Neural Networks, 10(5), 1048-1054.

[2] Graham, P. (2002). A plan for spam. Retrieved from paulgraham.com/spam.html

[3] Sahami, M., Dumais, S., Heckerman, D., & Horwitz, E. (1998). A Bayesian approach to filtering junk e-mail. AAAI Workshop on Learning for Text Categorization.

[4] Cormack, G. V. (2008). Email spam filtering: A systematic review. Foundations and Trends in Information Retrieval, 1(4), 335-455.

[5] Sculley, D., & Wachman, G. M. (2007). Relaxed online SVMs for spam filtering. Proceedings of the 30th Annual International ACM SIGIR Conference.

[6] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.

[7] scikit-learn developers. (2024). scikit-learn: Machine Learning in Python. Retrieved from scikit-learn.org

[8] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

[9] Streamlit Inc. (2024). Streamlit: The fastest way to build data apps. Retrieved from streamlit.io

[10] Anti-Phishing Working Group (APWG). (2024). Phishing Activity Trends Report Q2 2024.

[11] FBI Internet Crime Complaint Center (IC3). (2023). Internet Crime Report 2023.

[12] Statista Research Department. (2024). Spam email statistics worldwide 2024.

[13] Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 10(3), 61-74.

[14] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

[15] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

APPENDIX A DATASET SAMPLE ENTRIES

The following table presents a representative sample of emails from the dataset, covering each major category. Full dataset is available in dataset/ham_diverse.csv.

Table 9 Dataset sample entries

Category	Label	Email Text (Excerpt)
E-Commerce	ham	Your Amazon order #112-4857291-3847562 has shipped. Your package containing the Kindle Paperwhite case is on its way and should arrive by Thursday...
Workplace	ham	Hi Jess, your performance review with your manager David has been scheduled for November 5 at 2 PM in Conference Room B...
DevOps	ham	Your CI pipeline on GitHub Actions has completed successfully. Branch: feature/payment-refactor. All 214 tests passed. Coverage: 91.3%...
Education	ham	Hi students, Assignment 4 on dynamic programming is now available on the course portal. It is due November 12 at 11:59 PM...
Healthcare	ham	Hi Yuki, this is a reminder that you have an appointment with Dr. Sullivan on November 6 at 10:30 AM at North Side Family Medical...
Travel	ham	Hi Chris, just confirming your flight Delta 847 from Los Angeles to New York JFK on November 5 is confirmed. Departure is at 7:15 AM...
Personal	ham	Hey Sophie! It was so great running into you at the farmers market on Saturday. We should definitely plan to grab coffee soon...
Shipping	ham	Your DoorDash order from Chipotle has been picked up by your dasher. Estimated arrival is 7:43 PM. You can track your delivery in real time...

APPENDIX B SCAM PATTERN TAXONOMY

The following table documents the scam pattern taxonomy implemented in the rule-based scam detection engine, covering 9 primary scam categories and their key detection signals.

Table 10 Scam pattern taxonomy

Scam Category	Common Variants	Key Detection Signals
Advance Fee Fraud	419 scam, Nigerian prince, unclaimed inheritance	Promises large sums of money; requests upfront fees or bank details; uses exotic foreign locations
Phishing Attack	Credential phishing, account verification, fake login	Impersonates known brands; requests username/password; contains suspicious login links
Tech Support Scam	Fake Microsoft/Apple support, virus alert, remote access	Claims computer is infected; provides phone numbers; requests remote access or payment
Investment Fraud	Crypto pump-and-dump, guaranteed returns, fake trading platform	Promises unrealistic returns; creates urgency to invest; uses financial jargon
Romance Scam	Fake relationship, emotional manipulation, financial request	Unsolicited romantic contact; moves quickly to financial requests; uses military/foreign worker persona
Prize/Lottery Scam	Fake lottery win, sweepstakes fraud, gift card scam	Claims unsolicited prize win; requests taxes/fees before claiming; uses official-sounding names
Job Offer Scam	Work-from-home fraud, money mule, fake HR recruitment	Unsolicited job offer; pays before working; requests bank details for 'payroll setup'
Impersonation	IRS, FBI, bank, government agency fraud	Claims to be official agency; creates legal threats; demands immediate payment or personal data
Subscription Trap	Free trial with hidden recurring charges, fake renewal notice	Offers free trial with unclear terms; sends fake renewal invoices; demands cancellation fees